

- **PREFACE**
- **OBJECTIVE OF THE PROJECT**
- **HARDWARE AND SOFTWARE
REQUIREMENT**
- **ABOUT PYTHON**
- **ABOUT CSV**
- **ABOUT MATPLOTLIB**
- **SOURCE CODE**
- **OUTPUT**
- **CSV FILES OVERVIEW**
- **TOOLS/PLATFORMS AND LANGUAGES TO
BE USED**
- **CONCLUSION**
- **BIBLIOGRAPHY**

PREFACE

This project, "Sports Management Software," has been developed as part of my Class 12 Informatics Practices coursework. The primary objective of this project is to create an efficient and user-friendly software solution for managing various aspects of sports activities, including player records, team management, event scheduling, and performance tracking.

Through this project, I have explored key programming concepts in Python and utilized powerful libraries such as Pandas, NumPy, Matplotlib, and Seaborn to analyze and visualize sports-related data. The project also integrates data handling techniques to ensure smooth and efficient management of information.

The development of this project has not only enhanced my coding and analytical skills but has also provided valuable insights into data-driven decision-making in the field of sports management. It has helped me understand the importance of organizing and processing large datasets to derive meaningful conclusions.

I sincerely hope that this project serves as a useful demonstration of how technology can simplify and enhance sports administration. I have put in my best efforts to make this project functional, informative, and well-structured. Any suggestions for further improvement are always welcome.

Tejeshwar, Muhammed Omair
Class 12- E,D
Springdales Dubai

OBJECTIVE OF THE PROJECT

- ❶ **Player Management** – Maintain records of players, teams, and performance statistics.
- ❷ **Event Scheduling** – Organize and manage match schedules, tournaments, and fixtures
- ❸ **Performance Analysis** – Use data visualization tools like **Matplotlib and Seaborn** to analyze player and team statistics.
- ❹ **User-Friendly Interface** – Ensure a simple and efficient interface for managing sports-related data.
- ❺ **Data Handling & Security** – Use **Pandas and NumPy** for data processing and ensure secure storage of records.

This software aims to simplify sports administration by providing an efficient system for managing teams, matches, and analytics. Let me know if you want any modifications!

HARDWARE AND SOFTWARE REQUIREMENT

- **HARDWARE REQUIREMENT**

-> Hard Disk: 20 GB

-> RAM: 128 MB or more

-> Processor: Intel Core i3

-> Generation: 8th

-> Monitor: Any

• SOFTWARE REQUIREMENT

-> Operating System: Windows 7

-> Language: Python 3.6.2

-> Front End: IDLE(Python 3.6.2)

-> Back End: CSV Files

PYTHON:

Introduction:

Python is a **high-level, interpreted programming language** that is widely used for web development, data science, artificial intelligence, automation, and more. It was created by **Guido van Rossum** and first released in **1991**.

Features Of Python:

☑ **Simple & Easy to Learn** – Python has a clean and readable syntax, making it beginner-friendly.

☑ **Interpreted Language** – Python code is executed line-by-line, which makes debugging easier.

☑ **Cross-Platform** – Python works on **Windows, macOS, and Linux** without modification.

☑ **Extensive Libraries** – Includes powerful libraries like **NumPy, Pandas, Matplotlib, TensorFlow**, and more.

☑ **Object-Oriented & Dynamic** – Supports object-oriented, procedural, and functional programming styles.

Applications Of Python:

Web Development

Data Science & Machine Learning

Game Development

Cybersecurity & Ethical Hacking

Automation & Scripting

Python is one of the most in-demand programming languages today, making it an essential skill for students and professionals alike.

HOW TO INSTALL PYTHON

Step 1: Download Python

Go to the official Python website: <https://www.python.org/downloads/>

Click **Download Python [Latest Version]** The installer (.exe file) will start downloading

Step 2: Install Python

Open the downloaded .exe file.

Check the box "**Add Python to PATH**".

Click **Install Now** and wait for the installation to complete.

Click **Close** when done.

Click **Close** when done.

COMMA-SEPERATED VALUES

CSV File Introduction:

A CSV (Comma-Separated Values) file is a simple and widely used format for storing and exchanging data. It organizes information in a tabular format, where each row represents a record, and columns are separated by commas. CSV files are easy to create and read using spreadsheet programs like Microsoft Excel, Google Sheets, or even plain text editors. They are commonly used for data storage, database migration, and transferring information between different software applications. Since CSV files are lightweight and universally supported, they are an efficient way to handle structured data in various fields, including business, finance, and data analysis.

Advantages of CSV Files:

1. **Simple and Easy to Use** – Readable by both humans and machines.
2. **Lightweight and Compact** – Consumes minimal storage space.

3. **Wide Compatibility** – Supported by most applications, including Excel and databases.
4. **Fast Processing** – Quick to read and write due to its plain-text format.

MATPLOTLIB

Introduction:

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is widely used in data analysis, scientific computing, and machine learning to create visualizations such as plots, charts, and graphs.

Key Features of Matplotlib:

- Comprehensive Plotting Tools
- Customization and Flexibility
- Integration with Other Libraries
- Support for Interactive Plots
- Exporting to Multiple Formats
- Subplots and Grid Layouts
- Animations
- 3D Plotting
- Text and Annotations
- Cross-Platform Support

SOURCE CODE

```
if a == 1:

print(Fore.GREEN + "\tSports")

    print("""
    1) Add a new Sport in Academy
    2) Modify an Existing Sport in Academy
    3) Remove an Existing Sport in Academy
    4) Fetch Records of Sport in Academy
    """)

    sport1 = int(input(Fore.CYAN + "Choose Your Desired Option:
"))

    if sport1 == 1:

        print(Fore.YELLOW + "\tAdding a New Sport in Academy")

        sid = input(Fore.MAGENTA + "Enter Sports ID: ")

        while sid in sport.index:

            print(Fore.RED + 'Sport ID already exists!')

            sid = input(Fore.MAGENTA + "Enter a new Sports ID: ")

        sname = input(Fore.MAGENTA + "Enter Sports Name: ")

        sduration = input(Fore.MAGENTA + "Enter Time/Duration the
sport will last long: ")

        stype = input(Fore.MAGENTA + "Enter Sports Type: ")

    sport.loc[sid] = [sname, sduration, stype]

    print(Fore.GREEN + "New Sport Added:")
```

```

        print(sport)
    elif sport1 == 2:
        print(Fore.YELLOW + "\tModifying an Existing Sport in
Academy")

        print(Fore.CYAN + "Things you can Modify:")
        print("""
1. Sport Name
2. Duration
3. Type
""")
        sid2 = input(Fore.MAGENTA + "Enter Sports ID: ")
        if sid2 not in sport.index:
            print(Fore.RED + "No such Sport exists. Check your
Sports ID.")

            continue # Skip further processing if ID is invalid
        sport2 = int(input(Fore.CYAN + "Select the desired option
from the above menu: "))
        if sport2 == 1:
            sport_name = input(Fore.MAGENTA + "Enter New Sport
Name: ")

            sport.loc[sid2, "Sport Name"] = sport_name
            print(Fore.GREEN + "Data Changed Successfully")
        elif sport2 == 2:
            duration_t = input(Fore.MAGENTA + "Enter New
Duration/Time: ")

            sport.loc[sid2, "Duration"] = duration_t
            print(Fore.GREEN + "Data Changed Successfully")
    elif sport2 == 3:

```

```

        type_t = input(Fore.MAGENTA + "Enter New Type of
Sport: ")

        sport.loc[sid2, "Type"] = type_t

        print(Fore.GREEN + "Data Changed Successfully")

        print(sport)

        input(Fore.CYAN + "Press Enter to continue: ")

    elif sport1 == 3:

        print(Fore.YELLOW + "\tRemoving an Existing Sport in
Academy")

        sid3 = input(Fore.MAGENTA + "Enter Sports ID: ")

        if sid3 not in sport.index:

            print(Fore.RED + "No Such Sports Exists")

            input(Fore.CYAN + "Press Enter to continue: ")

            continue

        sportdel1 = input(Fore.MAGENTA + "Are you sure you want to
delete this sport? (y/n): ")

        if sportdel1.lower() == 'y':

            sport.drop(sid3, axis=0, inplace=True)

            print(Fore.GREEN + "Sport Removed Successfully")

            print(sport)

        else:

            print(Fore.RED + "Sport Not Removed")

            input(Fore.CYAN + "Press Enter to continue: ")

    elif sport1 == 4:

        print(Fore.YELLOW + "\tFetching Records")

print("""

1. All Records

2. Top Records

```

3. Bottom Records

4. Customized Records

```
""")
```

```
fetch1 = int(input(Fore.CYAN + "Enter the desired option  
from the above menu: "))
```

```
if fetch1 == 1:
```

```
    print(sport)
```

```
elif fetch1 == 3: n = int(input(Fore.CYAN + "How many records from the  
bottom you want to fetch? ")) tail = sport.tail(n) print(Fore.GREEN +  
"Bottom Records:") print(tail)
```

```
elif fetch1 == 4:
```

```
    print("""
```

```
        1. Indoor
```

```
        2. Outdoor
```

```
        3. Both
```

```
""")
```

```
fetch2 = int(input(Fore.CYAN + "Enter the desired option  
from the above menu: "))
```

```
if fetch2 == 1:
```

```
    indoorcond = sport['Type'] == 'Indoor'
```

```
    print(Fore.GREEN + "Indoor Sports:")
```

```
    print(sport[indoorcond])
```

```
elif fetch2 == 2:
```

```
    outdoorcond = sport['Type'] == 'Outdoor'
```

```
    print(Fore.GREEN + "Outdoor Sports:")
```

```
    print(sport[outdoorcond])
```

```
elif fetch2 == 3:
```

```
    print(Fore.GREEN + "Both Indoor and Outdoor Sports:")
```

```
    print(sport)
```

```
elif a == 2:
```

```
    print(Fore.GREEN + "\tCoaches")
```

```
    print("""
```

```
1) Add a new Coach in Academy
```

- 2) Modify an existing Data of Coach in Academy
 - 3) Remove an existing Coach from Academy
 - 4) Graphical Representation
 - 5) Fetch Records of Coach in Academy
- """)

```
sport3 = int(input(Fore.CYAN + "Choose Your Desired Option: "))
```

```
if sport3 == 1:
```

```
    print(Fore.YELLOW + "\tAdding a New Coach in Academy")
```

```
    coach_id = input(Fore.MAGENTA + "Enter Coach ID: ")
```

```
    while coach_id in coach.index:
```

```
        print(Fore.RED + 'Coach ID already exists!')
```

```
        coach_id = input(Fore.MAGENTA + "Enter a new Coach ID: ")
```

```
    cname = input(Fore.MAGENTA + "Enter Coach Name: ")
```

```
    age = int(input(Fore.MAGENTA + "Enter Coach Age: "))
```

```
    gender = input(Fore.MAGENTA + "Enter Coach Gender: ")
```

```
    sportsid = input(Fore.MAGENTA + "Enter Sports ID: ")
```

```
    emailid = input(Fore.MAGENTA + "Enter Academy E-Mail ID of
```

```
Coach: ")
```

```
    mnumber = int(input(Fore.MAGENTA + "Enter Mobile Number of
```

```
Coach: "))
```

```
    password = input(Fore.MAGENTA + "Enter Email ID Password of
```

```
Coach: ")
```

```
    salary = int(input(Fore.MAGENTA + "Enter Salary of Coach: "))
```

```
    rating = float(input(Fore.MAGENTA + "Enter Public Rating of
```

```
Coach: "))
```

```
    # Add the new coach to the DataFrame
```

```
    coach.loc[coach_id] = [cname, age, gender, sportsid, emailid,
mnumber, password, salary, rating]
```

```
    print(Fore.GREEN + "New Coach Added in Academy")
```

```
    print(coach)
```

```
    input(Fore.CYAN + "Press any key to continue: ")
```

```
elif sport3 == 2:
```

```
    print(Fore.YELLOW + "\tModifying an Existing Data of Coach in
```

```

Academy")
    print(Fore.CYAN + "Things you can modify:")
    print("""
1. Coach Name
2. Coach Age
3. Gender
4. Sports ID
5. Email ID
6. Mobile Number
7. Salary
8. Rating
""")

    modify_choice = int(input(Fore.CYAN + "Choose your desired
modification option: "))
    coach_id = input(Fore.MAGENTA + "Enter Coach ID: ")

    if coach_id not in coach.index:
        print(Fore.RED + "No such Coach exists. Check your Coach
ID.")

        input(Fore.CYAN + "Press Enter to continue: ")
        continue

    if modify_choice == 1:
        new_name = input(Fore.MAGENTA + "Enter New Coach Name: ")
        coach.loc[coach_id, "Coach Name"] = new_name
        print(Fore.GREEN + "Coach Name Updated")

    elif modify_choice == 2:
        new_age = int(input(Fore.MAGENTA + "Enter New Coach Age:
"))

        coach.loc[coach_id, "Age"] = new_age
        print(Fore.GREEN + "Coach Age Updated")

    elif modify_choice == 3:
        new_gender = input(Fore.MAGENTA + "Enter New Gender: ")
        coach.loc[coach_id, "Gender"] = new_gender
        print(Fore.GREEN + "Coach Gender Updated")

```

```

elif modify_choice == 4:
    new_sports_id = input(Fore.MAGENTA + "Enter New Sports ID: ")

    coach.loc[coach_id, "Sport ID"] = new_sports_id
    print(Fore.GREEN + "Coach Sports ID Updated")

elif modify_choice == 5:
    new_email = input(Fore.MAGENTA + "Enter New Email ID: ")
    coach.loc[coach_id, "Email ID"] = new_email
    print(Fore.GREEN + "Coach Email ID Updated")

elif modify_choice == 6:
    new_mobile = int(input(Fore.MAGENTA + "Enter New Mobile
Number: "))
    coach.loc[coach_id, "Mobile Number"] = new_mobile
    print(Fore.GREEN + "Coach Mobile Number Updated")

elif modify_choice == 7:
    new_salary = int(input(Fore.MAGENTA + "Enter New Salary: "))
    coach.loc[coach_id, "Salary"] = new_salary
    print(Fore.GREEN + "Coach Salary Updated")

elif modify_choice == 8:
    new_rating = float(input(Fore.MAGENTA + "Enter New Rating: "))
    coach.loc[coach_id, "Rating"] = new_rating
    print(Fore.GREEN + "Coach Rating Updated")

print(coach)
input(Fore.CYAN + "Press Enter to continue: ")

elif sport3 == 3:
    print(Fore.YELLOW + "\tRemoving an Existing Coach from
Academy")
    coach_id = input(Fore.MAGENTA + "Enter Coach ID: ")

    if coach_id not in coach.index:
        print(Fore.RED + "No such Coach exists. Check your Coach

```

```

ID.")
        input(Fore.CYAN + "Press Enter to continue: ")
        continue

        delete_confirm = input(Fore.MAGENTA + "Are you sure you want
to delete this coach? (y/n): ")
        if delete_confirm.lower() == 'y':
            coach.drop(coach_id, axis=0, inplace=True)
            print(Fore.GREEN + "Coach Removed Successfully")
        else:
            print(Fore.RED + "Coach Not Removed")

        print(coach)
        input(Fore.CYAN + "Press Enter to continue: ")

    elif sport3 == 4:
        print(Fore.YELLOW + "\tGraphical Representation")
        if 'Salary' in coach.columns:
            coach['Salary'].plot(kind='bar')
            plt.title("Coach Salary Distribution")
            plt.xlabel("Coach ID")
            plt.ylabel("Salary")
            plt.show()
        else:
            print(Fore.RED + "No salary data available for graphical
representation.")
            input(Fore.CYAN + "Press Enter to continue: ")

    elif sport3 == 5:
        print(Fore.YELLOW + "\tFetching Records of Coaches")
        print("""
1. All Records
2. Top Records
3. Bottom Records
4. Customized Records
""")

        fetch_choice = int(input(Fore.CYAN + "Choose your desired
option: "))

```



```

        if fetch_choice == 1:
            print(coach)

        elif fetch_choice == 2:
            n = int(input(Fore.CYAN + "How many top records do you
want to fetch? "))
            top_records = coach.head(n)
            print(Fore.GREEN + "Top Records:")
            print(top_records)

```

Ensuring proper syntax and structure

```

print("No Such Coach Exist") print("Check Your Coach ID") input("Press Enter to continue:
")

```

Corrected part for updating coach details

```

coach_id = input("Enter Coach ID: ")

if sport4 == 1: coach_name = input("Enter New Name of Coach: ")
coach.loc[coach_id, "Coach Name"] = coach_name print(coach)
print("Data Changed Successfully") input("Press Enter to continue:")

elif sport4 == 2: coach_age = int(input("Enter New Age of Coach: "))
coach.loc[coach_id, "Age"] = coach_age print(coach) print("Data
Changed Successfully") input("Press Enter to continue:")

elif sport4 == 3: coach_emailid = input("Enter New E-Mail ID of Coach:
") coach.loc[coach_id, "E-Mail ID"] = coach_emailid print(coach)
print("Data Changed Successfully") input("Press Enter to continue:")

elif sport4 == 4: coach_mnumber = int(input("Enter New Mobile Number
of Coach: ")) coach.loc[coach_id, "Mobile Number"] = coach_mnumber
print(coach) print("Data Changed Successfully") input("Press Enter to
continue:")

```

```

elif sport4 == 5: coach_password = input("Enter New Password of Coach:
") coach.loc[coach_id, "Password"] = coach_password print(coach)
print("Data Changed Successfully") input("Press Enter to continue:")

elif sport4 == 6: coach_salary = int(input("Enter New Salary of Coach:
")) coach.loc[coach_id, "Salary"] = coach_salary print(coach)
print("Data Changed Successfully") input("Press Enter to continue:")
import matplotlib.pyplot as plt

```

Assuming coach is already a DataFrame with 'Age' column

Example code for choosing the chart type

```

charttype = int(input("Enter the chart type (1 for Line Chart, 2 for
Bar Chart, 3 for Horizontal Bar Chart, 4 for another chart): "))

if charttype == 1: n = int(input("How many ages from the top do you
want to plot? ")) head1 = coach.head(n) age2 = head1['Age'] cindex =
head1.index plt.plot(cindex, age2, label="Coach Age's", color='green',
linewidth=4) plt.title("Line Chart Representing the Age of Coaches")
plt.xlabel("Coach ID") plt.ylabel("Age") plt.legend() plt.show()

elif charttype == 2: n = int(input("How many ages from the top do you
want to plot? ")) head1 = coach.head(n) age2 = head1['Age'] cindex =
head1.index plt.bar(cindex, age2, label="Coach Age's", color='blue')
plt.title("Bar Chart Representing the Age of Coaches")
plt.xlabel("Coach ID") plt.ylabel("Age") plt.legend() plt.show()

elif charttype == 3: n = int(input("How many ages from the top do you
want to plot? ")) head1 = coach.head(n) age2 = head1['Age'] cindex =
head1.index plt.barh(cindex, age2, label="Coach Age's",
color='violet') plt.title("Horizontal Bar Chart Representing the Age
of Coaches") plt.xlabel("Coach ID") plt.ylabel("Age") plt.legend()
plt.show()

```

```

elif charttype == 4: n = int(input("How many ages from the top do you
want to plot? ")) head1 = coach.head(n) age2 = head1['Age'] cindex =
head1.index # Add any other desired chart type here. Example:
plt.scatter(cindex, age2, label="Coach Age's", color='red')
plt.title("Scatter Chart Representing the Age of Coaches")
plt.xlabel("Coach ID") plt.ylabel("Age") plt.legend() plt.show()
import matplotlib.pyplot as plt

```

Assuming coach is a DataFrame with 'Salary' column

Example code for selecting the chart type

```

print("\nChoose Graph Type") print("=" * 30) print("1. Line Chart")
print("2. Vertical Bar Chart") print("3. Horizontal Bar Chart")
print("4. Histogram")

```

```

charttype2 = int(input("Select the desired chart type from the above
menu: "))

```

```

if charttype2 == 1: n = int(input("How many Salaries from the top you
want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.plot(cindex, salary2, label="Coach Salaries",
color='green', linewidth=2) plt.title("Line Chart Representing the
Salaries of Coaches") plt.xlabel("Coach ID") plt.ylabel("Salaries")
plt.legend() plt.show()

```

```

elif charttype2 == 2: n = int(input("How many Salaries from the top
you want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.bar(cindex, salary2, label="Coach Salaries",
color='orange') plt.title("Vertical Bar Chart Representing the
Salaries of Coaches") plt.xlabel("Coach ID") plt.ylabel("Salaries")
plt.legend() plt.show()

```

```

elif charttype2 == 3: n = int(input("How many Salaries from the top
you want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.barh(cindex, salary2, label="Coach Salaries",

```

```

color='blue') plt.title("Horizontal Bar Chart Representing the
Salaries of Coaches") plt.xlabel("Coach ID") plt.ylabel("Salaries")
plt.legend() plt.show()

elif charttype2 == 4: n = int(input("How many Salaries from the top
you want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.hist(salary2, bins=10, label="Coach
Salaries", color='purple') plt.title("Histogram Representing the
Salaries of Coaches") plt.xlabel("Salaries") plt.ylabel("Frequency")
plt.legend() plt.show() import matplotlib.pyplot as plt

```

Assuming coach is a DataFrame with 'Salary' column

Example code for selecting the chart type

```

print("\nChoose Graph Type") print("=" * 30) print("1. Line Chart")
print("2. Vertical Bar Chart") print("3. Horizontal Bar Chart")
print("4. Histogram")

charttype2 = int(input("Select the desired chart type from the above
menu: "))

if charttype2 == 1: n = int(input("How many Salaries from the top you
want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.plot(cindex, salary2, label="Coach Salaries",
color='green', linewidth=2) plt.title("Line Chart Representing the
Salaries of Coaches") plt.xlabel("Coach ID") plt.ylabel("Salaries")
plt.legend() plt.show()

elif charttype2 == 2: n = int(input("How many Salaries from the top
you want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.bar(cindex, salary2, label="Coach Salaries",
color='orange') plt.title("Vertical Bar Chart Representing the
Salaries of Coaches") plt.xlabel("Coach ID") plt.ylabel("Salaries")
plt.legend() plt.show()

```

```
elif charttype2 == 3: n = int(input("How many Salaries from the top
you want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.barh(cindex, salary2, label="Coach Salaries",
color='blue') plt.title("Horizontal Bar Chart Representing the
Salaries of Coaches") plt.xlabel("Coach ID") plt.ylabel("Salaries")
plt.legend() plt.show()
```

```
elif charttype2 == 4: n = int(input("How many Salaries from the top
you want to plot? ")) head1 = coach.head(n) salary2 = head1['Salary']
cindex = head1.index plt.hist(salary2, bins=10, label="Coach
Salaries", color='purple') plt.title("Histogram Representing the
Salaries of Coaches") plt.xlabel("Salaries") plt.ylabel("Frequency")
plt.legend() plt.show() import matplotlib.pyplot as plt
```

Assuming coach is a DataFrame with 'Ratings' column

Example code for selecting the chart type

```
print("\nChoose Graph Type") print("=" * 30) print("1. Line Chart")
print("2. Vertical Bar Chart") print("3. Horizontal Bar Chart")
print("4. Histogram")
```

```
charttype2 = int(input("Select the desired chart type from the above
menu: "))
```

```
if charttype2 == 1: n = int(input("How many Ratings from the top you
want to plot? ")) head1 = coach.head(n) rating2 = head1['Ratings']
cindex = head1.index plt.plot(cindex, rating2, label="Coach Ratings",
linewidth=4, color='crimson') plt.title("Line Chart Representing the
Ratings of Coaches") plt.xlabel("Coach ID") plt.ylabel("Ratings")
plt.legend() plt.show()
```

```
elif charttype2 == 2: n = int(input("How many Ratings from the top you
want to plot? ")) head1 = coach.head(n) rating2 = head1['Ratings']
cindex = head1.index plt.bar(cindex, rating2, label="Coach Ratings",
color='darkslategrey') plt.title("Vertical Bar Chart Representing the
```

```
Ratings of Coaches") plt.xlabel("Coach ID") plt.ylabel("Ratings")  
plt.legend() plt.show()
```

```
elif charttype2 == 3: n = int(input("How many Ratings from the top you  
want to plot? ")) head1 = coach.head(n) rating2 = head1['Ratings']  
cindex = head1.index plt.barh(cindex, rating2, label="Coach Ratings",  
color='indigo') plt.title("Horizontal Bar Chart Representing the  
Ratings of Coaches") plt.xlabel("Coach ID") plt.ylabel("Ratings")  
plt.legend() plt.show()
```

```
elif charttype2 == 4: n = int(input("How many Ratings from the top you  
want to plot? ")) head1 = coach.head(n) rating2 = head1['Ratings']  
cindex = head1.index plt.hist(rating2, bins=10, label="Coach Ratings",  
color='seagreen') plt.title("Histogram Representing the Ratings of  
Coaches") plt.xlabel("Ratings") plt.ylabel("Frequency") plt.legend()  
plt.show() import matplotlib.pyplot as plt
```

Assuming 'coach' is a DataFrame with 'Ratings' column

Fetching records and plotting ratings

Chart type selection and plotting ratings

```
if charttype2 == 4: n = int(input("How many Ratings from the top you  
want to plot? ")) head1 = coach.head(n) rating2 = head1['Ratings']  
cindex = head1.index plt.hist(rating2, bins=4, label="Coach Ratings",  
color='orangered', edgecolor='k') plt.title("Histogram Representing  
the Ratings of Coaches") plt.xlabel("Coach ID") plt.ylabel("Ratings")  
plt.legend() plt.show()
```

Fetching records

```
elif sport3 == 5: print("\tFetching Records") print("=" * 30) print("1. All Records") print("2.
Top Records") print("3. Bottom Records") print("4. Customized Records")

fetch2 = int(input("Enter the desired option from the above menu: "))

if fetch2 == 1:
    print(coach)
elif fetch2 == 2:
    n = int(input("How many records from the top you want to fetch?
"))
    head = coach.head(n)
    print(head)
elif fetch2 == 3:
    n = int(input("How many records from the bottom you want to fetch?
"))
    tail = coach.tail(n)
    print(tail)
elif fetch2 == 4:
    print("Choose record filter:")
    print("=" * 30)
    print("1. Age")
    print("2. Gender")
    print("3. Ratings")

    fetch3 = int(input("Enter the desired option from the above menu:
"))

    if fetch3 == 1:
        filter_value = input("Enter the age filter criteria (e.g.,
'Age > 30'): ")
        filtered_records = coach.query(filter_value)
        print(filtered_records)
    elif fetch3 == 2:
        gender = input("Enter gender (Male/Female): ")
        filtered_records = coach[coach['Gender'] == gender]
        print(filtered_records)
```

```

elif fetch3 == 3:
    filter_value = input("Enter the ratings filter criteria (e.g.,
'Ratings >= 4'): ")
    filtered_records = coach.query(filter_value)
    print(filtered_records)

```

Assuming coach is a DataFrame, and sport4 indicates the action (like 1 for Name, 2 for Age, etc.)

```

coach_id = input("Enter Coach ID: ")

if coach_id not in coach.index: print("No Such Coach Exists")
print("Check Your Coach ID") input("Press Enter to continue: ") else:
if sport4 == 1: coach_name = input("Enter New Name of Coach: ")
coach.loc[coach_id, "Coach Name"] = coach_name print() print(coach)
print("Data Changed Successfully") input("Press Enter to continue:")

elif sport4 == 2:
    coach_age = int(input("Enter New Age of Coach: "))
    coach.loc[coach_id, "Age"] = coach_age
    print()
    print(coach)
    print("Data Changed Successfully")
    input("Press Enter to continue:")

elif sport4 == 3:
    coach_emailid = input("Enter New E-Mail ID of Coach: ")
    coach.loc[coach_id, "E-Mail ID"] = coach_emailid
    print()
    print(coach)
    print("Data Changed Successfully")
    input("Press Enter to continue:")

elif sport4 == 4:

```



```
    coach_mnumber = int(input("Enter New Mobile Number of Coach: "))
    coach.loc[coach_id, "Mobile Number"] = coach_mnumber
    print()
    print(coach)
    print("Data Changed Successfully")
    input("Press Enter to continue:")

elif sport4 == 5:
    coach_password = input("Enter New Password of Coach: ")
    coach.loc[coach_id, "Password"] = coach_password
    print()
    print(coach)
    print("Data Changed Successfully")
    input("Press Enter to continue:")

elif sport4 == 6:
    coach_salary = float(input("Enter New Salary of Coach: "))
    coach.loc[coach_id, "Salary"] = coach_salary
    print()
    print(coach)
    print("Data Changed Successfully")
    input("Press Enter to continue:")

import matplotlib.pyplot as plt
```

Assuming 'coach' is a predefined DataFrame with 'Salary' column

Example: coach = pd.DataFrame({'Salary': [50000, 60000, 70000, 80000, 90000]}, index=[1, 2, 3, 4, 5])

```
sport5 = int(input("Enter a value for sport5: ")) # Ensure sport5 is defined
```

```
if sport5 == 2: print() print("\tChoose Graph Type") print("1. Line Chart") print("2. Vertical Bar Chart") print("3. Horizontal Bar Chart") print("4. Histogram")
```

```
charttype2 = int(input("Select the desired chart type from the above menu: "))
```

```
if charttype2 in [1, 2, 3, 4]:
```

```
    n = int(input("How many Salaries from the top do you want to plot? "))
```

```
    if n > len(coach):
        print(f"Only {len(coach)} records are available. Plotting all available data.")
```

```
        n = len(coach)
```

```
    head1 = coach.head(n)
```

```
    salary2 = head1['Salary']
```

```
    cindex = head1.index
```

```
    if charttype2 == 1:
```

```
        plt.plot(cindex, salary2, label="Coach Salaries", linewidth=3, color='b')
```

```
        plt.title("Line Chart Representing the Salaries of Coaches")
```

```

        plt.xlabel("Coach ID")
        plt.ylabel("Salaries")

    elif charttype2 == 2:
        plt.bar(cindex, salary2, label="Coach Salaries",
color='orange')
        plt.title("Bar Chart Representing the Salaries of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Salaries")

    elif charttype2 == 3:
        plt.barh(cindex, salary2, label="Coach Salaries",
color='green')
        plt.title("Horizontal Bar Chart Representing the Salaries of
Coaches")
        plt.xlabel("Salaries")
        plt.ylabel("Coach ID")

    elif charttype2 == 4:
        plt.hist(salary2, bins=5, label="Coach Salaries",
color='purple', edgecolor='black')
        plt.title("Histogram Representing the Salaries of Coaches")
        plt.xlabel("Salary Ranges")
        plt.ylabel("Frequency")

    plt.legend()
    plt.show()
else:
    print("Invalid chart type!")

import matplotlib.pyplot as plt

sport5 = int(input("Enter a value for sport5: ")) # Ensure sport5 is
defined

if sport5 == 2: print("\n\tChoose Graph Type") print("1. Line Chart")
print("2. Vertical Bar Chart") print("3. Horizontal Bar Chart")
print("4. Histogram")

```

```

charttype2 = int(input("Select the desired chart type from the above
menu: "))

if charttype2 in [1, 2, 3, 4]:
    n = int(input("How many Salaries from the top do you want to plot?
"))

    if n > len(coach):
        print(f"Only {len(coach)} records are available. Plotting all
available data.")
        n = len(coach)

    head1 = coach.head(n)
    salary2 = head1['Salary']
    cindex = head1.index

    if charttype2 == 1:
        plt.plot(cindex, salary2, label="Coach Salaries", linewidth=3,
color='b')
        plt.title("Line Chart Representing the Salaries of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Salaries")

    elif charttype2 == 2:
        plt.bar(cindex, salary2, label="Coach Salaries",
color='orange')
        plt.title("Bar Chart Representing the Salaries of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Salaries")

    elif charttype2 == 3:
        plt.barh(cindex, salary2, label="Coach Salaries",
color='green')
        plt.title("Horizontal Bar Chart Representing the Salaries of
Coaches")
        plt.xlabel("Salaries")
        plt.ylabel("Coach ID")

    elif charttype2 == 4:

```

```

        plt.hist(salary2, bins=4, label="Coach Salaries",
color='yellow', edgecolor='black')
        plt.title("Histogram Representing the Salaries of Coaches")
        plt.xlabel("Salary Ranges")
        plt.ylabel("Frequency")

    plt.legend()
    plt.show()
else:
    print("Invalid chart type!")

elif sport5 == 3: print("\n\tChoose Graph Type") print("1. Line
Chart") print("2. Vertical Bar Chart") print("3. Horizontal Bar
Chart") print("4. Histogram")

charttype2 = int(input("Select the desired chart type from the above
menu: "))

if charttype2 in [1, 2, 3, 4]:
    n = int(input("How many Ratings from the top do you want to plot?
"))

    if n > len(coach):
        print(f"Only {len(coach)} records are available. Plotting all
available data.")
        n = len(coach)

    head1 = coach.head(n)
    rating2 = head1['Ratings']
    cindex = head1.index

    if charttype2 == 1:
        plt.plot(cindex, rating2, label="Coach Ratings", linewidth=4,
color='crimson')
        plt.title("Line Chart Representing the Ratings of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Ratings")

    elif charttype2 == 2:

```

```

        plt.bar(cindex, rating2, label="Coach Ratings",
color='darkslategrey')
        plt.title("Bar Chart Representing the Ratings of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Ratings")

    elif charttype2 == 3:
        plt.barh(cindex, rating2, label="Coach Ratings", color='blue')
        plt.title("Horizontal Bar Chart Representing the Ratings of
Coaches")
        plt.xlabel("Ratings")
        plt.ylabel("Coach ID")

    elif charttype2 == 4:
        plt.hist(rating2, bins=4, label="Coach Ratings",
color='purple', edgecolor='black')
        plt.title("Histogram Representing the Ratings of Coaches")
        plt.xlabel("Rating Ranges")
        plt.ylabel("Frequency")

    plt.legend()
    plt.show()
else:
    print("Invalid chart type!")

```

```

import matplotlib.pyplot as plt

```

```

sport5 = int(input("Enter a value for sport5: ")) # Ensure sport5 is
defined

```

```

if sport5 == 3: print("\n\tChoose Graph Type") print("1. Line Chart")
print("2. Vertical Bar Chart") print("3. Horizontal Bar Chart")
print("4. Histogram")

```

```

charttype2 = int(input("Select the desired chart type from the above
menu: "))

```

```

if charttype2 in [1, 2, 3, 4]:
    n = int(input("How many Ratings from the top do you want to plot?

```

```

"))

    if n > len(coach):
        print(f"Only {len(coach)} records are available. Plotting all
available data.")
        n = len(coach)

    head1 = coach.head(n)
    rating2 = head1['Ratings']
    cindex = head1.index

    if charttype2 == 1:
        plt.plot(cindex, rating2, label="Coach Ratings", linewidth=4,
color='crimson')
        plt.title("Line Chart Representing the Ratings of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Ratings")

    elif charttype2 == 2:
        plt.bar(cindex, rating2, label="Coach Ratings",
color='darkslategrey')
        plt.title("Bar Chart Representing the Ratings of Coaches")
        plt.xlabel("Coach ID")
        plt.ylabel("Ratings")

    elif charttype2 == 3:
        plt.barh(cindex, rating2, label="Coach Ratings", color='blue')
        plt.title("Horizontal Bar Chart Representing the Ratings of
Coaches")
        plt.xlabel("Ratings")
        plt.ylabel("Coach ID")

    elif charttype2 == 4:
        plt.hist(rating2, bins=4, label="Coach Ratings",
color='orangered', edgecolor='black')
        plt.title("Histogram Representing the Ratings of Coaches")
        plt.xlabel("Rating Ranges")
        plt.ylabel("Frequency")

```

```

plt.legend()
plt.show()
else:
    print("Invalid chart type!")

elif sport5 == 5: print("\n\tFetching Records") print("1. All
Records") print("2. Top Records") print("3. Bottom Records") print("4.
Customized Records")

fetch2 = int(input("Enter the desired option from the above Menu: "))
if fetch2 == 1:
    print("\nAll Coach Records:")
    print(coach)

elif fetch2 == 2:
    n = int(input("How Many Records from the Top do you want to fetch?
"))
    if n > len(coach):
        print(f"Only {len(coach)} records are available. Displaying
all records.")
        n = len(coach)
        head = coach.head(n)
        print("\nTop Records:")
        print(head)

elif fetch2 == 3:
    n = int(input("How Many Records from the Bottom do you want to
fetch? "))
    if n > len(coach):
        print(f"Only {len(coach)} records are available. Displaying
all records.")
        n = len(coach)
        tail = coach.tail(n)
        print("\nBottom Records:")
        print(tail)

elif fetch2 == 4:
    print("\n\tCustomized Records Filter")
    print("1. Age")

```



```

print("2. Gender")
print("3. Ratings")

fetch3 = int(input("Enter the desired option from the above Menu:
"))

if fetch3 == 1:
    age_filter = int(input("Enter the minimum age to filter: "))
    filtered_data = coach[coach['Age'] >= age_filter]
    print("\nFiltered Records by Age:")
    print(filtered_data)

elif fetch3 == 2:
    gender_filter = input("Enter the gender to filter (M/F):
").strip().upper()
    filtered_data = coach[coach['Gender'] == gender_filter]
    print("\nFiltered Records by Gender:")
    print(filtered_data)

elif fetch3 == 3:
    rating_filter = float(input("Enter the minimum rating to
filter: "))
    filtered_data = coach[coach['Ratings'] >= rating_filter]
    print("\nFiltered Records by Ratings:")
    print(filtered_data)

else:
    print("Invalid option!")

else:
    print("Invalid option!")

```

Filtering by Age

```

print("\n\tFilter by Age") print("1. Greater than 35") print("2. Less than 35") print("3. Equal to
35")

```

```
ageinput = int(input("Enter the desired option: "))

if ageinput == 1: great35 = coach['Age'] > 35 print("\nCoaches older than 35:")
print(coach.loc[great35])

elif ageinput == 2: less35 = coach['Age'] < 35 print("\nCoaches younger than 35:")
print(coach.loc[less35])

elif ageinput == 3: eq35 = coach['Age'] == 35 print("\nCoaches who are exactly 35 years
old:") print(coach.loc[eq35])

else: print("Invalid option!")
```

Filtering by Gender

```
print("\n\tFilter by Gender") print("1. Male") print("2. Female")

geninput = int(input("Enter the desired option: "))

if geninput == 1: male = coach['Gender'] == 'Male' print("\nMale
Coaches:") print(coach.loc[male])

elif geninput == 2: female = coach['Gender'] == 'Female'
print("\nFemale Coaches:") print(coach.loc[female])

else: print("Invalid option!")
```

Filtering by Ratings

```
print("\n\tFilter by Ratings") print("1. Greater than 3.75") print("2.
Less than 3.75") print("3. Equal to 3.75")

ratinginput = int(input("Enter the desired option: "))

if ratinginput == 1: great375 = coach['Ratings'] > 3.75
print("\nCoaches with ratings greater than 3.75:")
print(coach.loc[great375])
```

```

elif ratinginput == 2: less375 = coach['Ratings'] < 3.75
print("\nCoaches with ratings less than 3.75:")
print(coach.loc[less375])

elif ratinginput == 3: eq375 = coach['Ratings'] == 3.75
print("\nCoaches with ratings exactly 3.75:") print(coach.loc[eq375])

else: print("Invalid option!") print("\n\tPlayer Management System")
print("1. Add a new Player in Academy") print("2. Modify an existing
Player's Data in Academy") print("3. Remove an existing Player from
Academy") print("4. Graphical Representation") print("5. Fetch Records
of Players in Academy")

sport6 = int(input("Choose Your Desired Option: "))

if sport6 == 1: print("\n\tAdding a New Player in Academy")

pid = input("Enter Player ID: ")
while pid in players.index:
    print("Player ID already exists! Please enter a unique ID.")
    pid = input("Enter Player ID: ")

pname = input("Enter Player Name: ")
address = input("Enter Player Address: ")
age = int(input("Enter Player Age: "))
gender = input("Enter Player Gender (Male/Female): ")
sportsid = input("Enter Sports ID: ")
emailid = input("Enter Academy E-Mail ID of Player: ")
mnumber = input("Enter Mobile Number of Player: ")
transportation = input("Enter Mode of Transportation: ")
password = input("Enter Mail ID Password of Player: ")

players.loc[pid] = [pname, address, age, gender, sportsid, emailid,
mnumber, transportation, password]

print("\nNew Player Added in Academy!")
print(players)
input("Press any key to continue...")

```

```

elif sport6 == 2: print("\n\tModifying an Existing Player's Data in
Academy") print("\nThings you can modify:") print("1. Player Name")
print("2. Address") print("3. Age") print("4. Sports ID") print("5. E-
Mail ID")

modify_option = int(input("Enter the field you want to modify: "))

pid = input("Enter Player ID to modify: ")
if pid not in players.index:
    print("Player ID not found!")
else:
    if modify_option == 1:
        new_name = input("Enter new Player Name: ")
        players.at[pid, "Name"] = new_name

    elif modify_option == 2:
        new_address = input("Enter new Address: ")
        players.at[pid, "Address"] = new_address

    elif modify_option == 3:
        new_age = int(input("Enter new Age: "))
        players.at[pid, "Age"] = new_age

    elif modify_option == 4:
        new_sportsid = input("Enter new Sports ID: ")
        players.at[pid, "Sports ID"] = new_sportsid

    elif modify_option == 5:
        new_emailid = input("Enter new E-Mail ID: ")
        players.at[pid, "E-Mail ID"] = new_emailid

    else:
        print("Invalid option!")

print("\nUpdated Player Data:")
print(players.loc[pid])
input("Press any key to continue...")

```

```
import matplotlib.pyplot as plt
```

Get user input for chart type

```
charttype = int(input("Select the desired chart type from the above menu: "))
```

```
if charttype == 1: n = int(input("How many Ages from the top do you want to plot? ")) age2 =  
players['Age'].head(n) pindex = players.index[:n]
```

```
plt.plot(pindex, age2, label="Players' Ages", linewidth=3,  
color='blue')  
plt.title("Line Chart Representing the Age of Players")  
plt.xlabel("Player ID")  
plt.ylabel("Age")  
plt.legend()  
plt.show()
```

```
elif charttype == 2: n = int(input("How many Ages from the top do you want to plot? "))  
head_ = players.head(n) age2 = head_['Age'] pindex = head_.index
```

```
plt.bar(pindex, age2, label="Players' Ages", color='yellowgreen')  
plt.title("Bar Chart Representing the Age of Players")  
plt.xlabel("Player ID")  
plt.ylabel("Age")  
plt.legend()  
plt.show()
```

```
elif charttype == 3: n = int(input("How many Ages from the top do you want to plot? "))  
head1 = players.head(n) age2 = head1['Age'] pindex = head1.index
```

```
plt.barh(pindex, age2, label="Players' Ages", color='midnightblue')  
plt.title("Horizontal Bar Chart Representing the Age of Players")  
plt.xlabel("Age")  
plt.ylabel("Player ID")  
plt.legend()  
plt.show()
```

```
elif charttype == 4: n = int(input("How many Ages from the top do you want to plot? ")) age2  
= players['Age'].head(n)
```

```
plt.hist(age2, bins=5, label="Players' Ages", color='orange',  
edgecolor='black')  
plt.title("Histogram Representing the Age of Players")  
plt.xlabel("Age Ranges")  
plt.ylabel("Frequency")  
plt.legend()  
plt.show()
```

```
else: print("Invalid chart type!") import matplotlib.pyplot as plt
```

Horizontal Bar Chart

```
x = y.index plt.barh(x, y, color="darkcyan") plt.ylabel("Count of Players")  
plt.xlabel("Transportation") plt.title("Transportation-wise Count of Players") plt.show()
```

Fetching Records Menu

```
print("\n\tFetching Records") print("""
```

1. All Records
2. Top Records
3. Bottom Records
4. Customized Records """)

```
fetch3 = int(input("Enter the desired option from the above menu: "))
```

```
if fetch3 == 1: print(players)
```

```
elif fetch3 == 2: n = int(input("How Many Records from the Top do you  
want to fetch? ")) head = players.head(n) print(head)
```

```
elif fetch3 == 3: n = int(input("How Many Records from the Bottom do  
you want to fetch? ")) tail = players.tail(n) print(tail)
```

```

elif fetch3 == 4: print("\n1. Age\n2. Gender\n3. Sports ID\n4.
Transportation") fetch4 = int(input("Enter the desired option from the
above menu: "))

if fetch4 == 1:
    print("\n1. Greater than 19\n2. Less than 19\n3. Equal to 19")
    ageinput = int(input("Enter the desired option: "))

    if ageinput == 1:
        great19 = players[players['Age'] > 19]
        print(great19)

    elif ageinput == 2:
        less19 = players[players['Age'] < 19]
        print(less19)
    elif ageinput == 3:
        eq19 = players[players['Age'] == 19]
        print(eq19)
    else:
        print("Invalid option!")
elif fetch4 == 2:
    print("\n1. Male\n2. Female")
    geninput = int(input("Enter the desired option: "))
    if geninput == 1:
        male = players[players['Gender'] == 'Male']
        print(male)

    elif geninput == 2:
        female = players[players['Gender'] == 'Female']
        print(female)
    else:
        print("Invalid option!")
elif fetch4 == 3:
    sports_id = input("Enter the Sports ID: ")
    filtered_players = players[players['Sports ID'] == sports_id]
    print(filtered_players)

elif fetch4 == 4:
    transport_mode = input("Enter the mode of transportation: ")

```

```

        transport_players = players[players['Transportation'] ==
transport_mode]
        print(transport_players)

else:
    print("Invalid option!")

```

```

else: print("Invalid option!")

```

Get the Sports ID from user

```

sportid11 = input("Enter Sports ID: ")

```

List of valid Sports IDs

```

valid_sport_ids = [ 'S013', 'S014', 'S015', 'S016', 'S017', 'S018',
'S019', 'S020', 'S021', 'S022', 'S023', 'S024', 'S025', 'S026',
'S027', 'S028', 'S029', 'S030', 'S031', 'S032' ]

```

Check if the entered Sports ID is valid

```

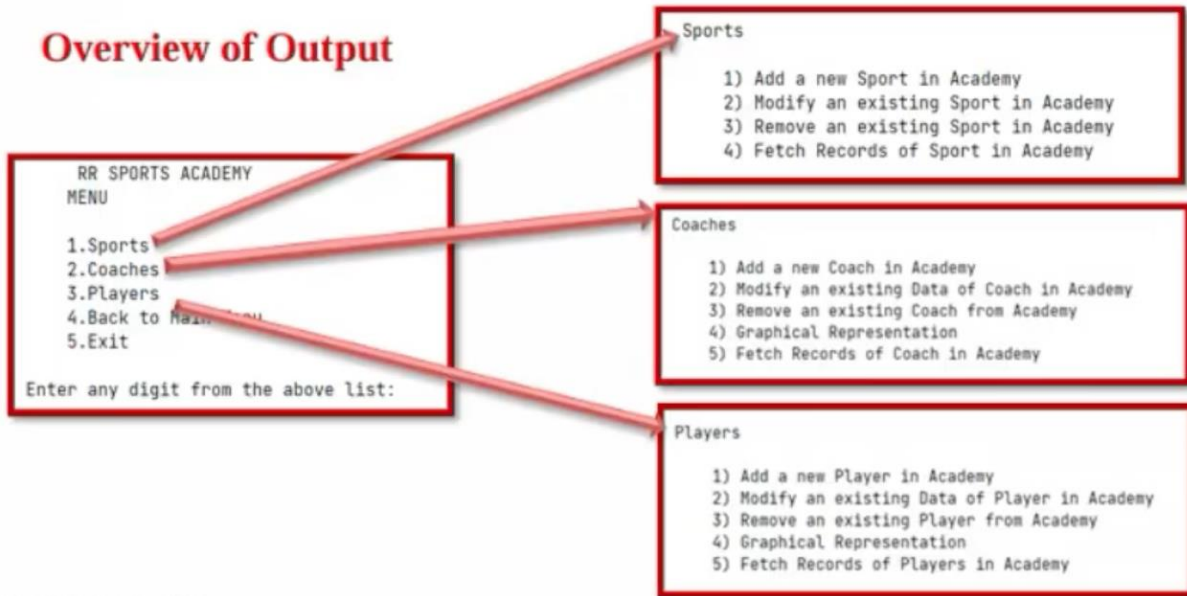
if sportid11 in valid_sport_ids: print(players.groupby('Sports
ID').get_group(sportid11)) else: print("Invalid Sports ID!") print("""

```

1. Self
2. Provided """ transportinput = int(input("Enter the desired option: ")) if transportinput == 1: self_transport = players['Transportation'] == 'Self' print(players[self_transport]) elif transportinput == 2: provided_transport = players['Transportation'] == 'Provided' print(players[provided_transport]) else: print("\tThank You!")

OUTPUT

Overview of Output



RR SPORTS ACADEMY

Enter your choice2

Coach ID	Coach Name	Age	Gen der	Sports ID \
C001	Rahul Dravid	46.0	Male	S001
C002	Batista Fernando	41.0	Female	S002
C003	Cadenas Montanes Manuel	38.0	Female	S003
C004	Ferraro Hernan	34.0	Female	S004
C005	Gomez Cora Santiago	37.0	Female	S005
C006	Hernandez Sergio	47.0	Female	S006
C007	Lopez Monica Susana	43.0	Female	S007
C008	Mendez Marcelo Rodolfo	59.0	Male	S008
C009	Retegui Carlos Jose	52.0	Male	S009
C010	Retegui Carlos Jose	41.0	Male	S010
C011	Rodriguez Eduardo Rafael	46.0	Male	S011
C012	Woelflin Roberto Osvaldo	40.0	Male	S012
C013	Arnold Graham	49.0	Male	S013
C014	Batch Colin	42.0	Male	S014
C015	Mohamed Mai	43.0	Male	S015
C016	Fuli Saiasi	45.0	Female	S016
C017	Davy Jeremy	45.0	Male	S017
C018	David Warner	45.0	Male	S018
C019	Cristiano Ronaldo	39.0	Male	S019

C026	Manenti John	34.0	Female	S026
C027	Arko	37.0	Male	S027
C028	Smriti Mandhana	38.0	Female	S028
C029	Neeraj Chopra	39.0	Male	S029
C030	M.S. Dhoni	32.0	Male	S030
C031	Saina Nehwal	31.0	Female	S031
C032	Rillie John	55.0	Female	S032
C033	Staniforth Dave	56.0	Female	S033
C034	Walsh Tim	52.0	Male	S034
C035	Brun Aristide	63.0	Male	S035
C036	Pradeep Narwal	37.0	Male	S008
C000	Akshit	34.0	Male	S003

Coach ID	E-Mail ID	Mobile Number	Password
C001	rahuldravid@rracademy.com	7.878654e+09	C0018425678345
C002	batistafernando@rracademy.com	6.485736e+09	C0026485736453
C003	cadenasmontanesmanuel@rracademy.com	6.492035e+09	C0036492034967
C004	ferrarohernan@rracademy.com	9.075343e+09	C0049075342617
C005	gomezcorasantiago@rracademy.com	9.530282e+09	C0059530281910
C006	hernandezsergio@rracademy.com	8.015343e+09	C0068015342891
C007	lopezmonicasusana@rracademy.com	9.024387e+09	C0079024387251
C008	mendezmarcelorodolfo@rracademy.com	9.328216e+09	C0089328216111
C009	reteguicarlosjose@rracademy.com	7.024365e+09	C0097024364924
C010	reteguicarlosjose@rracademy.com	6.938456e+09	C0106938456283
C011	rodriguezeduardorafael@rracademy.com	7.063453e+09	C0117063452912
C012	woelflinrobertoosvaldo@rracademy.com	9.274112e+09	C0129274112233
C013	arnoldgraham@rracademy.com	9.993452e+09	C0139993451782
C014	batchcolin@rracademy.com	8.162435e+09	C0148162435193
C015	mohamedmai@rracademy.com	7.291739e+09	C0157291739211
C016	fulisaiasi@rracademy.com	7.319385e+09	C0167319384621
C017	davyjeremy@rracademy.com	9.382483e+09	C0179382482910
C018	davidwarner@rracademy.com	9.025132e+09	C0189025132436
C019	cristianoronaldo@rracademy.com	9.143810e+09	C0199143810313
C020	mithaliraj@rracademy.com	6.232121e+09	C0206232121234
C021	tonystark@rracademy.com	9.823246e+09	C0219823245631
C022	scarlettjohnson@rracademy.com	8.643213e+09	C0228643213456
C023	bucky@rracademy.com	9.035473e+09	C0239035472910
C024	jonesnathan@rracademy.com	7.138139e+09	C0247138138901
C025	emmawatson@rracademy.com	6.384902e+09	C0256384902312
C026	manentijohn@rracademy.com	7.362185e+09	C0267362184902
C027	arko@rracademy.com	7.246219e+09	C0277246219475
C028	smritimandhana@rracademy.com	9.103135e+09	C0289103134521
C029	neerajchopra@rracademy.com	7.849462e+09	C0297849462194
C030	msdhoni@rracademy.com	8.930423e+09	C0308930423131
C031	sainanehwal@rracademy.com	6.027589e+09	C0316027589210
C032	rilliejohn@rracademy.com	7.012359e+09	C0327012359201
C000	akshit@rracademy.com	7.103456e+09	C0007103456000

C006	150000.0	5.00
C007	230000.0	3.00
C008	432000.0	3.00
C009	259000.0	3.00
C010	280000.0	3.00
C011	200000.0	4.00
C012	230000.0	3.00
C013	370000.0	4.00
C014	320000.0	4.00
C015	240000.0	5.00
C016	265000.0	3.00
C017	230000.0	4.00
C018	210000.0	4.00
C019	400000.0	5.00
C020	190000.0	3.00
C021	200000.0	5.00
C022	170000.0	5.00
C023	100000.0	5.00
C024	210000.0	4.00
C025	250000.0	3.00
C026	221000.0	5.00
C027	340000.0	4.00
C028	230000.0	3.00
C029	250000.0	4.00
C030	210000.0	5.00
C031	340000.0	5.00
C032	230000.0	3.00
C033	280000.0	3.00
C034	240000.0	5.00
C035	430000.0	5.00
C036	275000.0	3.75
C000	45000.0	4.50

MENU

- 1.Sports
- 2.Coaches
- 3.Players
- 4.Back to Main Menu
- 5.Exit

Enter any digit from the above list: 1

1

Sports

- 1) Add a new Sport in Academy
- 2) Modify an existing Sport in Academy
- 3) Remove an existing Sport in Academy
- 4) Fetch Records of Sport in Academy

Choose Your Desired Option: 4

1. All Records
2. Top Records

Enter the desired option from the above Menu: 3

Sports ID	Sport Name	Duration	Type
S003	Basketball	48 Minutes	Both
S005	Boccia	8 Minutes	Both
S010	Dance	NaN	Both
S019	Handball	30 Minutes	Both
S020	High Jump	NaN	Both
S022	Judo	5 Minutes	Both
S023	Kabaddi	40 Minutes	Both
S024	Karate	2 Minutes	Both
S026	Long Jump	NaN	Both
S029	Shooting	75 Minutes	Both
S030	Shot put	NaN	Both

- 1.Sports
- 2.Coaches
- 3.Players
- 4.Back to Main Menu
- 5.Exit

Enter any digit from the above list: 5

5

Thank You!

GRAPHICAL REPRESENTATION

2 RR SPORTS ACADEMY MENU

Coaches

1.Sports
2.Coaches
3.Players
4.Back to Main Menu
5.Exit

Enter any digit from the above list: 2

2
Coaches

1) Add a new Coach in Academy
2) Modify an existing Data of Coach in Academy
3) Remove an existing Coach from Academy
4) Graphical Representation
5) Fetch Records of Coach in Academy

Choose Your Desired Option: 4

Graphical Representation Menu

1. Age
2. Salary
3. Ratings

Select the desired option from the above menu:

RR SPORTS ACADEMY MENU

Players

1.Sports
2.Coaches
3.Players
4.Back to Main Menu
5.Exit

Enter any digit from the above list: 3

3
Players

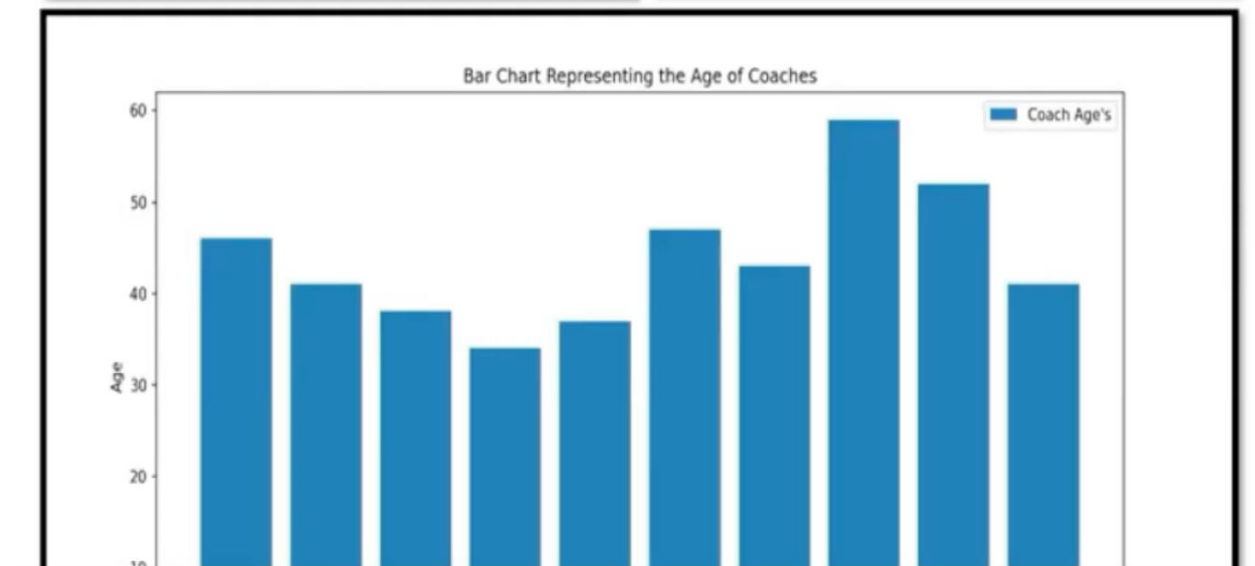
1) Add a new Player in Academy
2) Modify an existing Data of Player in Academy
3) Remove an existing Player from Academy
4) Graphical Representation
5) Fetch Records of Players in Academy

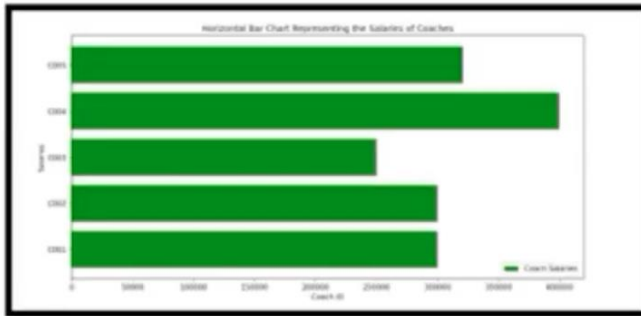
Choose Your Desired Option: 4

Graphical Representation Menu

1. Age
2. Gender
3. Transportation

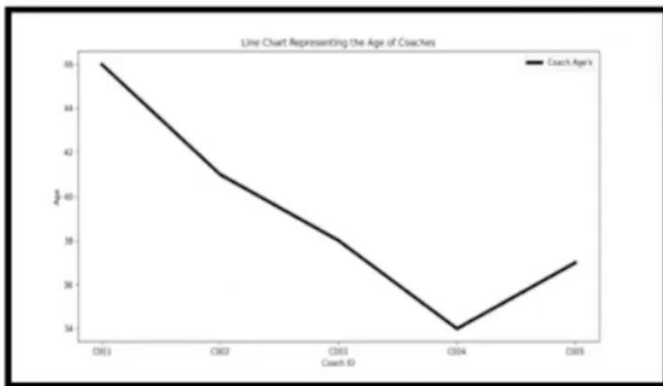
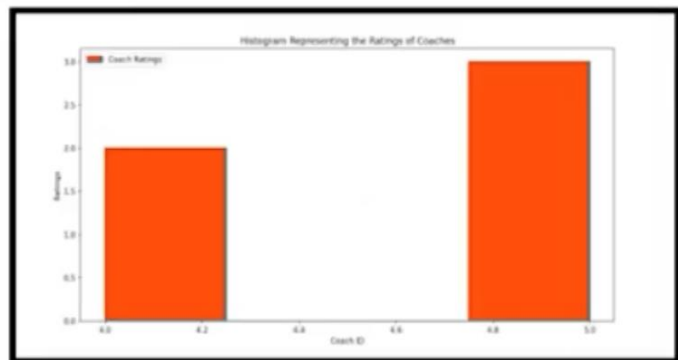
Select the desired option from the above menu: |





hBAR

Histogram



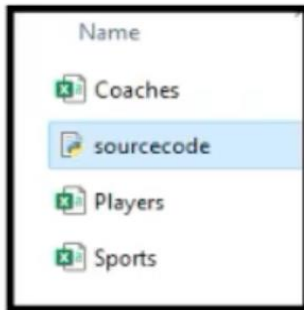
Line

CSV FILE OVERVIEW

```

sport=pd.read_csv("Sports.csv",index_col=0)
coach=pd.read_csv('Coaches.csv',index_col=0)
players=pd.read_csv('Players.csv',index_col=0)

```



Players.csv

	A	B	C	D	E	F	G	H	I	J
1	Player ID	Player Name	Address	Age	Gender	Sports ID	E-Mail ID	Mobile No.	Transport	Password
2	P001	Chris Gayl P2, Awad		17	Male	S006	jonathana	9.96E+09	Self	9.56E+09
3	P002	Darryl Wil Okay PLU		22	Male	S025	darrylwili	6.77E+09	Self	8.56E+09
4	P003	Mary Mil Surya Patl		23	Female	S028	marymills	8.56E+09	Provided	8.56E+09
5	P004	Ram gadga		34	Female	S034	shannonn	8.56E+09	Provided	8.56E+09
6	P005	Brian Harri E-194, Riik		15	Male	S020	brianharri	8.56E+09	Self	7.56E+09
7	P006	Christina 1197-198, E		15	Female	S031	christinaw	7.56E+09	Self	8.56E+09
8	P007	Mary Hun 153, Gand		21	Female	S011	maryhunt	8.56E+09	Provided	8.56E+09
9	P008	Richard W Surya Patl		23	Male	S023	richardwe	8.56E+09	Self	7.56E+09
10	P009	James Bro 172, Nagd		27	Male	S035	jamesbro	7.56E+09	Self	8.56E+09
11	P010	Ryan Mills 104, Achai		18	Male	S012	ryanmills	8.56E+09	Self	7.56E+09
12	P011	Stacy War 6, 381 - C,		27	Female	S002	stacyward	7.56E+09	Self	8.56E+09
13	P012	Jay Monte Shop No 3		20	Male	S019	jaymonte	8.56E+09	Self	8.56E+09
14	P013	Victor Jon Shop No 6		21	Male	S014	victorjone	8.56E+09	Self	7.56E+09
15	P014	Barbara S-214/a Mal		23	Female	S007	barbarasa	7.56E+09	Provided	9.96E+09
16	P015	Monica M Chemtex I		18	Female	S030	monicam	9.96E+09	Self	7.56E+09
17	P016	Michelle C 77, Sushe		22	Female	S003	michellegr	7.56E+09	Self	9.56E+09
18	P017	Michael A 47, Sushes		26	Male	S018	michaelal	9.56E+09	Provided	8.56E+09
19	P018	Cory Turn E-134, Riik		23	Male	S004	coryturne	8.56E+09	Self	9.36E+09
20	P019	Jeffrey Ha E-197, Riik		23	Female	S016	jeffreyhar	9.36E+09	Provided	9.56E+09
21	P020	Christoph-222, Nehri		19	Male	S008	christoph	9.56E+09	Provided	9.76E+09
22	P021	Denise Bri 321, Uttan		21	Female	S009	denisebro	9.76E+09	Provided	9.26E+09
23	P022	Laura Tayl F 95, Najal		16	Female	S005	laurataylo	9.26E+09	Self	9.06E+09
24	P023	Megan Sh 31, 3rd Mi		25	Female	S022	megansha	9.06E+09	Provided	8.56E+09
25	P024	Michael Si 82, Mittal		15	Male	S026	michaelst	8.56E+09	Self	9.96E+09
26	P025	Barbara B F 95, Najal		26	Female	S010	barbarabu	9.96E+09	Self	7.56E+09

Sports.csv

	A	B	C	D
1	Sports ID	Sport Name	Duration	Type
2	S001	Archery	2 Minutes	Outdoor
3	S002	Badminton	40-50 Mini	Indoor
4	S003	Basketball	48 Minute	Both
5	S004	Pool	56 Minute	Indoor
6	S005	Boccia	8 Minutes	Both
7	S006	Bowling	10 Minute	Indoor
8	S007	Boxing	36 Minute	Indoor
9	S008	Chess		Indoor
10	S009	Cricket		Outdoor
11	S010	Dance		Both
12	S011	Dodgeball	40 Minute	Outdoor
13	S012	Fencing	9 Minutes	Indoor
14	S013	Field Hock	35 Minute	Outdoor
15	S014	Football	90 Minute	Outdoor
16	S015	Frisbee	36 Minute	Indoor
17	S016	Futsal	40 Minute	Indoor
18	S017	Goalball	24 Minute	Indoor
19	S018	Golf		Outdoor
20	S019	Handball	30 Minute	Both
21	S020	High Jump		Both
22	S021	Ice Hockey	60 Minute	Outdoor
23	S022	Judo	5 Minutes	Both
24	S023	Kabaddi	40 Minute	Both
25	S024	Karate	2 Minutes	Both
26	S025	Lawn Tenr	90 Minute	Outdoor
27	S026	Long Jump		Both
28	S027	Polo	90-120 Min	Outdoor
29	S028	Rugby	80 Minute	Indoor
30	S029	Shooting	75 Minute	Both
31	S030	Shot put		Both
32	S031	Swimming		Indoor
33	S032	Table Teni	10 Minute	Indoor
34	S033	Taekwond	6 Minutes	Indoor
35	S034	Volleyball	60-90 Mini	Outdoor
36	S035	Weight Lifting		Indoor

Coaches.csv

	A	B	C	D	E	F	G	H	I	J
5	C004	Ferraro Hernan	34 Female	S004	ferrarohernan@rracad	9.08E+09	C0049075342617		400000	5
6	C005	Gomez Cora Santiago	37 Female	S005	gomezcorsantiago@rr	9.53E+09	C0059530281910		321000	5
7	C006	Hernandez Sergio	47 Female	S006	hernandezsergio@rrac	8.02E+09	C0068015342891		150000	5
8	C007	Lopez Monica Susana	43 Female	S007	lopezmonicasusana@r	9.02E+09	C0079024387251		230000	3
9	C008	Mendez Marcelo Rodolfo	59 Male	S008	mendezmarcelorodolf	9.33E+09	C0089328216111		432000	3
10	C009	Retegui Carlos Jose	52 Male	S009	reteguicarlosjose@rrac	7.02E+09	C0097024364924		259000	3
11	C010	Retegui Carlos Jose	41 Male	S010	reteguicarlosjose@rrac	6.94E+09	C0106938456283		280000	3
12	C011	Rodriguez Eduardo Rafael	46 Male	S011	rodriguezeduardorafae	7.06E+09	C0117063452912		200000	4
13	C012	Woelflin Roberto Osvaldo	40 Male	S012	woelflinrobertoosvald	9.27E+09	C0129274112233		230000	3
14	C013	Arnold Graham	49 Male	S013	arnoldgraham@rracad	9.99E+09	C0139993451782		370000	4
15	C014	Batch Colin	42 Male	S014	batchcolin@rracademy	8.16E+09	C0148162435193		320000	4
16	C015	Mohamed Mai	43 Male	S015	mohamedmai@rracad	7.29E+09	C0157291739211		240000	5
17	C016	Fuli Saiasi	45 Female	S016	fulisaiasi@rracademy.c	7.32E+09	C0167319384621		265000	3
18	C017	Davy Jeremy	45 Male	S017	davyjeremy@rracad	9.38E+09	C0179382482910		230000	4
19	C018	David Warner	45 Male	S018	davidwarner@rracad	9.03E+09	C0189025132436		210000	4
20	C019	Cristiano Ronaldo	39 Male	S019	cristianoronaldo@rraci	9.14E+09	C0199143810313		400000	5
21	C020	Mithali Raj	32 Female	S020	mithaliraj@rracademy	6.23E+09	C0206232121234		190000	3
22	C021	Tony Stark	38 Male	S021	tonystark@rracademy	9.82E+09	C0219823245631		200000	5
23	C022	Scarlett Johnson	30 Female	S022	scarlettjohnson@rraca	8.64E+09	C0228643213456		170000	5
24	C023	Bucky	35 Male	S023	bucky@rracademy.com	9.04E+09	C0239035472910		100000	5
25	C024	Jones Nathan	38 Female	S024	jonesnathan@rracader	7.14E+09	C0247138138901		210000	4
26	C025	Emma Watson	39 Female	S025	emmawatson@rracade	6.38E+09	C0256384902312		250000	3
27	C026	Manenti John	34 Female	S026	manentijohn@rracade	7.36E+09	C0267362184902		221000	5
28	C027	Arko	37 Male	S027	arko@rracademy.com	7.25E+09	C0277246219475		340000	4
29	C028	Smriti Mandhana	38 Female	S028	smritimandhana@rraci	9.1E+09	C0289103134521		230000	3
30	C029	Neeraj Chopra	39 Male	S029	neerajchopra@rracade	7.85E+09	C0297849462194		250000	4
31	C030	M.S. Dhoni	32 Male	S030	msdhoni@rracademy.c	8.93E+09	C0308930423131		210000	5
32	C031	Saina Nehwal	31 Female	S031	sainanehwal@rracade	6.03E+09	C0316027589210		340000	5
33	C032	Rillie John	55 Female	S032	rilliejohn@rracademy	7.01E+09	C0327012359201		230000	3
34	C033	Staniforth Dave	56 Female	S033	staniforthdave@rracac	7.1E+09	C0337103458902		280000	3
35	C034	Walsh Tim	52 Male	S034	walshtim@rracademy	9.84E+09	C0349836392010		240000	5
36	C035	Brun Aristide	63 Male	S035	brunaristide@rracader	9.12E+09	C0359123401934		430000	5
37	C036	Pradeep Narwal	37 Male	S008	pradeepnarwal@rracac	6.39E+09	C0366392100090		275000	3.75
38	C000	Akshit	34 Male	S003	akshit@rrsportsacad	4632746	58wyr		45000	4.5

CONCLUSION

Sports Management software offers functionalities to manage sports, coaches, and players, making it easier for administrators to keep track of all relevant information and perform essential tasks efficiently.

Through this project, the user can:

- **Add new sports** to the academy, ensuring the academy's offerings remain up-to-date.
- **Modify or remove existing sports**, providing flexibility in managing the academy's activities.
- **Maintain a record of coaches and players**, ensuring proper records are kept for easy reference.
- **View data in an organized manner**, making it easy to extract information whenever needed.

The project demonstrates the importance of database management, user interface design, and handling of dynamic data, all of which contribute to the efficient operation of a sports academy. The use of data structures and libraries like **Pandas** and **Matplotlib** also showcases the potential of data analysis and visualization for management purposes.

This software not only meets the requirements of the Class 12 IP curriculum but also presents an opportunity to learn real-world skills that are applicable in managing any data-driven system. Through this project, I have gained valuable experience in programming, problem-solving, and the practical application of theoretical concepts.

BIBLIOGRAPHY

- **Matplotlib Documentation** (2024). *Matplotlib - Python plotting library*. Retrieved from <https://matplotlib.org/>
- **Pandas Documentation** (2024). *Pandas - Python Data Analysis Library*. Retrieved from <https://pandas.pydata.org/>
- **GeeksforGeeks** (2024). *Python Programming Tutorials*. Retrieved from <https://www.geeksforgeeks.org/python-programming-language/>
- **Stack Overflow** (2024). *Stack Overflow - Community-driven Q&A platform for programmers*. Retrieved from <https://stackoverflow.com/>
- **W3Schools** (2024). *Learn Python Programming*. Retrieved from <https://www.w3schools.com/python/>
- **Real Python** (2024). *Python Programming Tutorials and Resources*. Retrieved from <https://realpython.com/>

